

18 — Un indice al contrario

La domanda a cui risponde: hai una mappa figlio → genitore e ti serve rispondere a genitore → figli. Cosa cambia davvero fra le due direzioni? E cosa fai quando la risposta è *trentatré*?

Per contorno: come si decide cosa mettere sotto una card quando la stessa griglia serve due viste diverse, e dove finisce il motore e comincia la rete.

Il fatto

Nei **Preferiti** ogni card aveva sotto i due pulsanti + e –, ereditati dal catalogo. Ma i preferiti non sono un inventario da aggiornare: sono le carte che ti piacciono, quelle che riguardi. Là quei due bersagli da 38 px non li tocca nessuno — anzi, si toccano per sbaglio scorrendo col pollice.

La domanda che invece viene sempre, guardando un Machoke, è un'altra: **ho anche il resto della linea?** Da qui il pulsante “Linea evolutiva” e la finestra che mostra la famiglia intera, con scritto su ogni gradino se ce l'hai.

Parte 1 — Le due direzioni di un indice

Il progetto ha già `data/evoluzioni.json`, prodotto da `tools/genera-indice-evoluzioni.mjs`:

```
{ "da": { "machoke": "Machop", "machamp": "Machoke" }, "nonPokemon": ["Fossile Raro", ...] }
```

È una mappa **figlio** → **genitore**. Risalire è banale: da machamp leggi Machoke, normalizzi, leggi ancora, e in tre passi sei al Base. Il modulo `engine/linee.js` faceva già esattamente questo per capire quali carte stampare.

Scendere è un'altra faccenda, e le differenze sono tre:

1. **Non c'è un accesso diretto.** Per sapere cosa evolve da Pikachu devi guardare *tutti* i valori. È una Map da costruire apposta:

```
function rovescia(indice) {
  const giu = new Map();
  for (const [evoluzione, preEvoluzione] of Object.entries(indice)) {
    const chiave = normalizzaNome(preEvoluzione);
    if (!giu.has(chiave)) giu.set(chiave, []);
    giu.get(chiave).push(evoluzione);
  }
  return giu;
}
```

In Java è la differenza fra una `Map<String, String>` e una `Map<String, List<String>>` — o, se hai lavorato con JPA, fra il lato `@ManyToOne` (che ha la colonna) e il lato `@OneToMany` (che non ce l'ha, e va ricostruito con una query).

2. **La cardinalità cambia.** Verso l'alto ogni Pokémon ha **una sola** pre-evoluzione: la catena è una lista. Verso il basso si dirama: sette nomi da Pikachu, **trentatré da Eevee**. Il tipo di ritorno non può essere lo stesso — infatti `catenaEvolutiva()` restituisce, per ogni livello, un *array* di nomi.
3. **Cambia la forma dei nomi.** E questo è il tranello. Nel file, i **valori** sono nomi come stanno sulla carta (Machop), le **chiavi** sono nomi normalizzati (raichu gx, minuscolo, senza trattini — vedi `engine/nomi.js`). Risalendo raccogli valori: nomi belli. Scendendo raccogli chiavi: nomi *tecnici*, che a schermo sarebbero brutti.

Non si è “sistemato” il file — un indice esiste per essere cercato, e le sue chiavi devono restare confrontabili. Si è deciso invece che quei nomi sono **buoni da cercare, non da stampare**: `app/linea-evolutiva.js` cerca la carta vera e ne usa il nome. Il JSDoc del modulo lo dichiara, perché è un contratto che nessun compilatore fa rispettare.

Il tetto, e perché non è un dettaglio estetico

Trentatré miniature in una finestra aperta per rispondere a “com’è fatta questa famiglia” non sono una risposta: sono un elenco. Quindi c’è un tetto per livello (8) e una riga che dichiara quanto resta fuori — «e altre 25 evoluzioni non mostrate». **Un dato tagliato in silenzio è peggio di un dato assente**: è la stessa regola per cui la ricerca globale dice “troppi risultati” invece di mostrarne dieci a caso.

Ma tagliare i primi otto in ordine d’indice dava questo:

Dark Espeon · Dark Flareon · Dark Jolteon · Dark Vaporeon · Espeon · Espeon ex · ...

Le quattro varianti “Dark” si prendevano metà schermo e **Flareon, Jolteon, Vaporeon restavano fuori**. Il primo rimedio è stato ordinare meglio prima di tagliare — le carte che possiedi davanti, poi i nomi corti, che è la stessa euristica della ricerca per nome. Migliorava le cose e **non era ancora la domanda giusta**: vedi la Parte 5, dove il tetto quasi non serve più.

Parte 2 — Dove passa il confine

Tre file nuovi, e nessuno dei tre potrebbe fare il lavoro di un altro:

File	Cosa sa	Cosa non può fare
src/engine/catena.js	l’algoritmo: risalire, rovesciare, tagliare	toccare la rete o il DOM
src/app/linea-evolutiva.js	dove si trovano le carte (collezione, catalogo)	disegnare
src/ui/linea-evolutiva/	come si disegna un gradino	sapere cos’è un indice

Il confine è quello di sempre in questo progetto (src/engine/ è puro), ma qui si vede *perché* conviene: la parte difficile — cicli nell’indice, cardinalità, tetti, ordinamento — è verificabile in tests/catena.test.js con sette righe di finto indice, senza browser, senza rete, senza IndexedDB. Il caso “il ventaglio si taglia a 3 e ne dichiara 9” è un test da mezzo secondo; a mano, sarebbe aprire l’app e contare le miniature di Eevee.

Nota il verso della dipendenza: linee.js — che esisteva già — ora **importa** da catena.js la risalita, invece di tenerne una copia. Due camminate sullo stesso indice che divergono col tempo darebbero due linee diverse per la stessa carta, e nessun test lo direbbe.

Parte 3 — Aprire prima di sapere

Ricostruire la linea di un Machoke vuol dire cercare Machop e Machamp in tutto il catalogo, cioè **scaricare il file di qualche set**. Sul telefono di casa sono secondi.

Se la finestra si aprisse solo a dati pronti, il tocco sembrerebbe non aver fatto nulla — e il secondo tocco arriverebbe subito dopo. Quindi il componente ha due momenti distinti:

```
finestra.apri(voce.carta.nome); // subito: titolo e "ricostruisco..."
finestra.gradini = (await struttura(...)).gradini; // i gradini, coi buchi
for (...) finestra.completa(livello, posizione, await dalCatalogo(nome));
```

(Tre momenti, in realtà, e il terzo è arrivato dopo: vedi la Parte 5.)

È lo stesso schema del *resolver* di Angular ribaltato: là la rotta aspetta i dati, qui la vista si apre e li accoglie. E come ogni cosa asincrona che l’utente può ripetere, ha bisogno di un guardiano contro le risposte sorpassate:

```
const mio = ++giro;
...
if (mio === giro) finestra.gradini = gradini;
```

Chiudi la linea di Machoke, apri quella di Eevee, e la risposta lenta della prima arriva dopo: senza il contatore, la finestra si riempirebbe della famiglia sbagliata. Lo stesso `#giroRicerca` c'è nella griglia per la ricerca per nome — quando un pattern compare due volte, è un pattern.

Parte 4 — Una griglia, due piedi

La vista Preferiti è la griglia del catalogo con `{preferito: 'solo'}` fisso (vedi il documento 17). Cambiare il piede della card senza duplicare il componente è una riga di decisione:

```
#piede(voce, mancante) {  
  if (!this.#fissi.preferito) return this.#stepper(voce, mancante);  
  if (voce.carta?.categoria !== 'Pokémon') return ''; // un'Energia non ha linea  
  return `<div class="piede-preferito">...</div>`;  
}
```

Due cose valgono più della riga in sé:

- **la funzionalità non si perde:** le copie restano modificabili aprendo la carta nel visore. Togliere un comando è legittimo solo se esiste ancora una strada;
- **il pulsante non compare dove non ha senso:** su un Allenatore o su un'Energia aprirebbe una finestra con dentro una carta sola. Un comando che non fa niente insegna a diffidare di tutti gli altri.

Parte 5 — Quattro difetti trovati usandola

La prima versione funzionava e aveva quattro difetti, tutti visti da chi la usava e non da chi la scriveva. Vale la pena guardarli insieme, perché nessuno è un errore di logica: sono quattro modi diversi di aver deciso troppo presto che un caso non esisteva.

«Manca lo zoom»

Una griglia di carte, in quest'app, si tocca per ingrandire. La finestra della linea mostrava carte e non si toccavano: chi la usa non ha imparato una nuova regola, ha pensato che mancasse un pezzo. Aveva ragione.

Il rimedio è di tre righe, ma la parte interessante è **chi apre il visore**: non questa finestra. Si annuncia carta-scelta, e risponde `app.js`, che è l'unico posto in cui si sa dove il visore stia:

```
this.dispatchEvent(new CustomEvent('carta-scelta', {  
  bubbles: true,  
  detail: { carta: voce.carta, nomeSet, lista, indice },  
}));
```

`lista` è tutta la linea, così dal Machop si passa al Machop con una frecciata. Ed è lo stesso evento che emette la griglia: un ascoltatore solo, su `document`, serve due sorgenti che non si conoscono.

Il visore però si apre **sopra** questa finestra: due `<dialog>` modali insieme. Funziona (il top layer li impila nell'ordine di apertura), ma ha fatto emergere un difetto in un modulo che stava lì da mesi: `blocca-scroll.js` aveva un solo interruttore, e chiudendo il visore la pagina tornava a scorrere **sotto** una finestra ancora aperta. Ora ogni pannello si presenta con una chiave:

```
export function sbloccaScorrimento(chiave = 'pannello') {  
  chiHaChiesto.delete(chiave);  
  if (chiHaChiesto.size) return; // qualcun altro lo tiene ancora  
  ...  
}
```

Un contatore sarebbe stato più corto e **sbagliato**: `chiudi()` del visore sblocca, e l'evento `close` che ne segue sblocca una seconda volta. Con un insieme la ripetizione non fa niente; con un contatore il conto sarebbe andato sotto zero. *Idempotente* batte *contato* ogni volta che il numero di chiamate non è sotto il tuo controllo.

«Lycanroc di notte sì, di giorno no»

Il difetto segnalato: la linea di Rockruff mostrava **quattro** caselle — Lycanroc, Lycanroc, Lycanroc-ex, Lycanroc GX — dove il gradino è uno.

La prima diagnosi è stata parziale. Guardando i due Lycanroc uguali si è visto un difetto vero: la collezione era indicizzata per nome tenendo, di ogni nome, **una sola** stampa — quella con più copie —, e chi possiede Forma Giorno e Forma Notte (due carte diverse, che nei dati si chiamano tutte e due Lycanroc: la forma non è scritta da nessuna parte) ne vedeva sparire una. Quello sì è corretto: le tue stampe si mostrano tutte.

Ma il difetto **principale** era un altro, ed è una domanda mal posta più che un bug. Rovesciando l'indice si ottengono i **nomi delle carte** che evolvono da Rockruff. Una linea evolutiva non è fatta di carte: è fatta di **specie**. Lycanroc-ex e Lycanroc GX non sono gradini, sono la stessa bestia stampata in modo speciale.

Quindi i nomi di un livello si accorpano prima di qualunque taglio:

```
const gruppo = specie.find(({ nome: capofila }) =>
  `${n} `.includes(` ${normalizzaNome(capofila)} `),
);
```

Tre cose di questa riga:

- **Nessun elenco di suffissi.** Sarebbe stato istintivo scrivere ['ex', 'gx', 'v', 'vmax', 'vstar', ' ', 'dark', 'mega', ...] e togliere quelli. È un elenco che invecchia a ogni espansione nuova. La regola qui è *strutturale*: ordinati dal più corto, un nome che ne contiene un altro già tenuto è una sua versione. Il nome corto è sempre la specie.
- **Gli spazi attorno** fanno il confine di parola in un colpo solo, e coprono i tre casi: Dark Espeon (davanti), Espeon ex (dietro), Mega Gardevoir ex (in mezzo — che senza il caso “in mezzo” restava fuori, ed è stato trovato provando la regola sull'indice vero).
- **Senza il confine**, Nidorina **si mangerebbe** Nidorino. È l'unico modo in cui questa regola può fare danno, e si prova in due righe di test.

L'effetto sul caso che aveva fatto arrabbiare il tetto:

	prima	dopo
Eevee, livello 1	33 nomi, 8 mostrati, «e altre 25»	8 specie , tutte
Rockruff, livello 1	4 caselle	1 specie (più le tue stampe)

Il tetto non è stato tolto — serve ancora se un giorno una specie ne generasse davvero nove — ma ha smesso di essere la difesa principale. **Accorpare non è tagliare**: oltre resta a zero, perché non manca niente.

Le varianti non si buttano: restano attaccate alla specie in *varianti*, e servono a rispondere “ce l'hai?” per chi di Lycanroc possiede solo il GX. La sua carta esiste e quel gradino lo occupa lei.

«Dark Crobat sta al gradino di Golbat»

Zubat □ Golbat □ Crobat. La finestra però metteva Dark Crobat accanto a Golbat: un Livello 2 in mezzo ai Livello 1.

Non era un difetto del codice. Nel dataset, la carta *Dark Crobat* (neo4 #2) è un Livello 2 e dichiara *evolveDa*: Zubat, che è il Base. L'indice riporta fedelmente quello che c'è scritto, e chi ricostruisce la linea leggendo solo da ottiene una linea sbagliata. (Non è un caso isolato: *Dark Golbat* in base5 #24 dichiara di evolvere da **Weavile**.)

Qui c'è una lezione che vale oltre questo progetto: **quando un dato esterno si contraddice, cerca il campo che permette di accorgersene**. La contraddizione era già tutta nella carta — stadio Livello 2, pre-evoluzione un Base — solo che nessuno la stava leggendo. Lo stadio è stampato sulla carta vera, *evolveDa* è un nome scritto a mano: fra i due, il primo è più affidabile.

Quindi `tools/genera-indice-evoluzioni.mjs` ora salva anche *stadi*, lo stadio di ogni specie come numero (0 Base, 1, 2), e `catena.js` lo usa come giudice:

```
const suo = stadi[chiave];
if (atteso !== null && Number.isInteger(suo) && suo !== atteso) {
  if (suo > atteso) rimandati.push({ nome: figlio, livello: suo });
  continue;
}
```

Tre dettagli che valgono più della regola:

- **Rimandare, non buttare.** Un nome arrivato troppo presto si mette da parte e torna al gradino suo. Dark Crobot riappare al livello 2, dove raggruppaPerSpecie() lo assorbe dentro Crobot — invisibile, ma se *quella* carta ce l'hai, il gradino dice “ce l'hai” invece di “non ce l'hai”.
- **Il punto fermo.** I gradini sono posizioni in un array, gli stadi sono un dato della carta: per confrontarli serve sapere a che stadio corrisponde il gradino zero. Lo dà lo stadio della carta di partenza. Se non si conosce (classifica() torna null), **non si corregge niente**: meglio l'indice così com'è che una correzione fatta a caso. C'è un test anche per questo.
- **La maggioranza.** Le stampe si contraddicono anche sullo stadio, quindi il generatore conta e vince lo stadio più frequente. Non è la verità, è la lettura più difendibile senza guardare 21.000 carte a una a una.

Costo: data/evoluzioni.json passa da 35 a 65 KB. Sta nel guscio del service worker, quindi si scarica sempre — è il prezzo di una linea giusta, e si paga una volta.

«Ogni tanto si blocca il touch»

Il difetto più istruttivo, perché non era visibile in nessuna delle prove fatte scrivendolo. La prima versione risolveva tutte le carte mancanti così:

```
await Promise.all(nomi.map(dalCatalogo)); // tutte insieme
```

Su Eevee sono nove ricerche in parallelo, e ogni ricerca poteva aprire fino a **dodici file di set** (è il tetto di cercaPerNomeGlobale, tarato per un uso diverso: identificare una carta fisica, dove vuoi vedere tutte le stampe). Ogni file che arriva è un JSON.parse da qualche megabyte, e JSON.parse è **sincrono**: il thread principale si ferma. Non abbastanza da sembrare un crash, abbastanza da mangiarsi i tocchi. *Si blocca e poi si riprende.*

Tre rimedi, in ordine di importanza:

1. **Prima la struttura, poi le carte.** La finestra riceve subito gradini e nomi — che sono l'informazione principale — e ogni carta arriva dopo, al posto del suo segnaposto. L'attesa percepita passa da nove secondi a zero.
2. **Una ricerca per volta**, non nove insieme. Il costo totale è lo stesso ma è spalmato: fra una e l'altra la pagina respira e raccoglie i tocchi.
3. **Un tetto più basso per questo uso:** cercaPerNomeGlobale ha imparato a ricevere maxSet. Qui ne bastano due — la domanda è *che faccia ha questo Pokémon*, e una stampa vale l'altra.

Il punto 3 merita una nota di progettazione: la funzione aveva un tetto giusto *per il suo primo chiamante*. Il secondo chiamante aveva bisogni diversi, e la scelta è stata **parametrizzare il tetto lasciando invariato il default**, non abbassarlo per tutti. Chi c'era prima non se ne accorge.

Domande di verifica

1. **La direzione mancante.** rovescia() ricostruisce la mappa a ogni apertura della finestra invece di tenerla in una variabile di modulo. Il commento dice che una cache renderebbe il modulo “non più puro”: cosa vuol dire in pratica? Scrivi un test che fallirebbe se la cache ci fosse.
2. **Il ciclo.** L'indice è un dato esterno, ricostruito da uno strumento. Se contenesse alfa → Beta e beta → Alfa, cosa succederebbe alla risalita senza il Set dei nomi già visti? E alla discesa? (Ce n'è un test.)
3. **Il tetto giusto.** Dopo l'accorpamento per specie, sul livello 1 di Eevee non resta fuori niente. Il tetto di 8 serve ancora a qualcosa? Cerca nei dati una specie che ne generi più di otto — e se non la trovi, di se toglieresti il tetto o lo lasceresti, e perché.

4. **Il tetto delle ricerche.** `MAX_RICERCHE = 12` in `app/linea-evolutiva.js` limita quante carte si vanno a cercare. Cosa vede l'utente per una carta oltre il dodicesimo posto? Guarda il ramo `trovata = null` e di' se ti sembra una degradazione onesta.
5. **Il verso dell'import.** `linee.js` importa da `catena.js`. Prova a immaginare il contrario — `catena.js` che importa da `linee.js` — e spiega perché sarebbe sbagliato guardando cosa altro si porterebbe dietro (`fabbisogno.js`, `stadi.js`, i punteggi dei mazzi).
6. **Il pannello sopra il pannello.** Apri la linea, ingrandisci una carta, chiudi il visore con `Esc` invece che col pulsante. Segui il codice: quante volte viene chiamato `sbloccaScorrimento` e con quali chiavi?
7. **Sincrono di nascosto.** `JSON.parse` di un file di set blocca il thread. Il progetto non usa Web Worker: quali altre operazioni dell'app hanno lo stesso profilo, e perché finora non si sono notate? (Guarda `caricaSet` e chi lo chiama.)
8. **Il nome come chiave.** Cerca nel progetto altri punti in cui il nome di una carta viene usato come identità. Per ciascuno, di' se il caso `Lycanroc` lo romperebbe o no, e perché.
9. **La regola strutturale.** `raggruppaPerSpecie()` accorpa senza conoscere né `ex` né `GX`. Scrivi due nomi di Pokémon veri che la ingannerebbero, poi verifica sull'indice (`data/evoluzioni.json`) se possono davvero comparire come figli dello stesso Pokémon. Se non possono, la regola è sbagliata lo stesso?
10. **Fidarsi dei dati.** `stadi` serve a smentire `da`. Cerca nel dataset altre coppie di campi in cui uno può smentire l'altro (per esempio `stadio` ed `evolveDa` sulla stessa carta, o `totale` e i numeri di collezione di un set) e di' quale dei due crederesti, e perché.